

Bloodrose — Modular Architecture Document

Governed Production Architecture — Structural Blueprint for Modular Decomposition

Project: Bloodrose

Artifact: Modular Architecture Document

Version: v0.1

Classification: Governed Production Architecture

Phase: Phase 2 — Modular Separation

Predecessor: Phase 1 Single-File HTML Prototype

Date: April 21, 2026

Status: Active

1. Architecture Philosophy

Bloodrose Phase 1 exists as a single-file HTML prototype — a monolithic artifact that proves the core game loop functions: the player navigates the living heart-garden, collects emotional petals, sustains thorn damage, and reaches the altar under the governance of corruption and redemption axes. Phase 1 validates the *what*. Phase 2 addresses the *how*. This document decomposes the monolith into governed, independently testable modules with explicit interfaces, transforming a proof-of-concept into a production-grade architecture capable of sustained development, audit, and extension.

The architecture follows **separation of concerns** as its foundational organizing principle. State management, game rules, event processing, entity definitions, rendering, and operator tooling are distinct modules that communicate through defined contracts — never through implicit coupling, shared mutable globals, or undocumented side channels. Each module has a single, well-defined purpose. Each

module's inputs and outputs are documented. Each module can be reasoned about, tested, and replaced without cascading failures across the system.

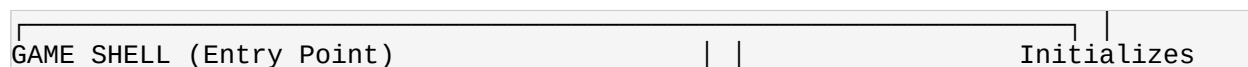
Determinism is non-negotiable. Every module must produce the same output given the same input. There is no hidden state. There are no random modifiers in v0.1. There are no side effects that bypass the State Manager. A recorded session replayed against the same module stack must produce identical state history, tick for tick, mutation for mutation. This constraint is not aspirational — it is a hard architectural requirement that enables regression testing, replay debugging, and canonical compliance verification.

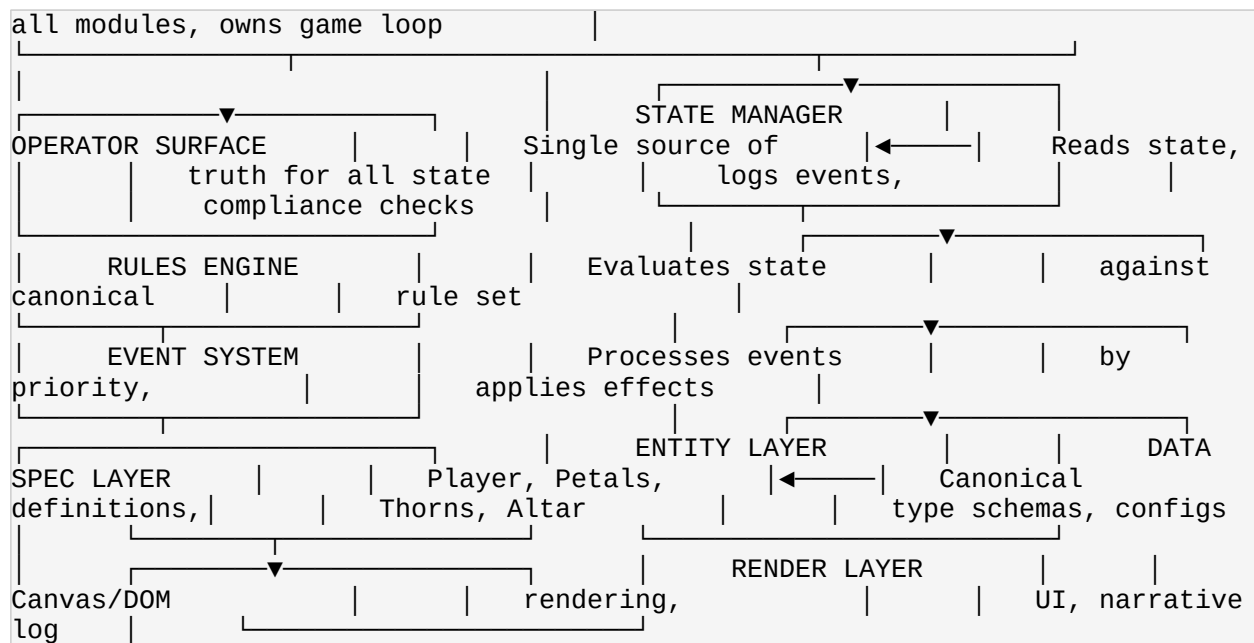
Governed extensibility is the mechanism by which Bloodrose grows. New mechanics, entities, events, or narrative branches are added through the canonical amendment process — not by hacking modules, introducing ad-hoc parameters, or circumventing interface contracts. The architecture must support growth without drift. Every extension point is defined. Every data constant is sourced from the Data Spec Layer. Every event maps to a canonical EVT-ID. The system is designed to be extended by contributors who have never seen the codebase, provided they follow the contracts defined in this document and the Contractor Contract artifact.

The architecture is designed for a browser-native runtime (HTML/CSS/JavaScript) but the module boundaries are language-agnostic in concept. The same separation of State Manager, Rules Engine, Event System, Entity Layer, Data Spec, Render Layer, and Operator Surface could be implemented in any runtime environment. The contracts defined here are structural, not syntactic. This document governs *what modules exist, what they do, how they communicate, and what they are forbidden from doing*.

2. Architecture Overview Diagram

The following diagram illustrates the major modules and their structural relationships. Data flows downward through the stack. State changes flow upward through the State Manager. No module may reach across layers to modify another module's internal state.





Architectural Invariant

All data flows downward through the stack. State changes flow upward through the State Manager. No module may reach across layers to modify another module's internal state. The Operator Surface sits alongside the Render Layer but serves a different audience — it observes the same data stream in raw, unglamoured form.

3. Module Definitions

Each module is defined by its purpose, responsibilities, inputs, outputs, dependencies, canonical references, and governance constraints. These definitions are normative — they define what each module *must* do and what it is *forbidden* from doing.

3.1 Game Shell

Purpose: Top-level entry point. Initializes all modules, owns the game loop (requestAnimationFrame or fixed-step tick), and coordinates module execution order.

Responsibilities

- Module instantiation and dependency injection
- Game loop timing and tick signal distribution
- Session lifecycle management (start, pause, resume, restart)
- Global error handling and fault containment

Interface

Direction	Description
Inputs	None (entry point)
Outputs	Tick signals to all modules
Dependencies	All other modules (instantiation only)

Governance Constraint

The Game Shell must not contain game logic. It is an orchestrator only. All rules, state, and events live in their respective modules. The Game Shell may reset state on session restart via the State Manager's `reset()` method, but it must never directly manipulate game state values.

3.2 State Manager

Purpose: Single source of truth for all game state. No other module may own or mutate state independently.

Responsibilities

- Stores and exposes all canonical state variables: `corruption`, `redemption`, `score`, `player_position`, `petal_instances`, `thorn_instances`, `game_over`, `ending_type`
- Provides getter/setter API with caller identification for audit trail

- Validates state mutations with bounds checking (corruption 0–100, redemption 0–100)
- Emits state-change notifications to subscribers
- Supports state snapshot for debugging, replay, and Operator Surface inspection

Interface

Direction	Description
Inputs	State mutation requests from Rules Engine and Event System (via setter API)
Outputs	Current state (via getter API); state-change events to subscribers
Dependencies	None (leaf dependency — no other module required)

Canonical References: Bloodrose Canonical Spec v0.1, Section 7 (Canonical State Model).

Governance Constraint

All state mutations must go through the setter API. Direct property assignment is forbidden. Setters must enforce bounds (clamp corruption/redemption to 0–100). Every mutation is logged with the calling module's identity for the Operator Surface audit trail. The State Manager must support serialization (JSON export) for snapshots. No shadow state, no local caches of mutable values anywhere in the system.

3.3 Rules Engine

Purpose: Evaluates the current game state against the canonical rule set and determines what should happen. The Rules Engine answers one question: *"Given this state, what conditions are met?"*

Responsibilities

- Collision detection: player vs. petals, player vs. thorns, player vs. altar
- Threshold evaluation: corruption ≥ 50 , ≥ 75 , ≥ 100 ; redemption ≥ 30 , ≥ 65
- Win/lose condition evaluation
- Pacing constraint evaluation (score milestones)

Interface

Direction	Description
Inputs	Current state from State Manager; entity positions from Entity Layer
Outputs	Condition flags and triggered rule IDs to Event System
Dependencies	State Manager (read-only); Entity Layer (position queries)

Canonical References: Bloodrose Canonical Spec v0.1, Sections 6, 9, 10.

Governance Constraint

The Rules Engine must not modify state directly. It evaluates and reports. The Event System applies effects. This separation is mandatory — it ensures auditability by making it possible to inspect what was *evaluated* independently of what was *applied*. The Rules Engine is a pure function: same inputs must always produce the same outputs.

3.4 Event System

Purpose: Processes events triggered by the Rules Engine, applies state effects, and dispatches narrative output. This is the execution arm of the Canonical Event Table.

Responsibilities

- Maintains the event registry (loaded from Canonical Event Table at initialization)
- Receives triggered conditions from Rules Engine
- Resolves event priority (Priority 0 terminal events override all others)
- Applies state effects through State Manager setters
- Dispatches narrative text to Render Layer narrative log
- Tracks one-shot events (threshold events that fire only once per session)
- Emits event-fired notifications to Operator Surface

Interface

Direction	Description
Inputs	Triggered condition flags from Rules Engine
Outputs	State mutations (via State Manager API); narrative text dispatches; event log entries
Dependencies	State Manager (write access via API); Rules Engine (receives triggers); Render Layer (narrative dispatch); Data Spec (event definitions)

Canonical References: Bloodrose Canonical Event Table v0.1 (entire document).

Critical Governance Constraint

Every event processed must have a matching EVT-ID in the Canonical Event Table. If an unregistered event attempts to fire, the system must log a CRITICAL alert and refuse to process it. The Event System is the *only* module authorized to request state mutations in response to gameplay (the Game Shell may reset state on session restart). Priority ordering must be deterministic: Priority 0 first, then 1, 2, 3, 4, 5, with stable ordering within the same priority level.

3.5 Entity Layer

Purpose: Defines and manages all game entities — their types, properties, positions, and lifecycle.

Responsibilities

- Entity type definitions: Player, GuiltPetal, DesirePetal, RagePetal, MercyPetal, ThornZone, Altar
- Entity instantiation and initial placement
- Position and bounds management
- Entity removal (petals consumed on collection)
- Entity queries (e.g., "all petals within radius," "is player in thorn zone")

Interface

Direction	Description
Inputs	Entity definitions from Data Spec Layer; position updates from game loop (player movement from input)
Outputs	Entity state queries (positions, bounds, active/inactive status) to Rules Engine and Render Layer
Dependencies	Data Spec Layer (entity schemas and configurations)

Canonical References: Bloodrose Canonical Spec v0.1, Sections 4, 5, 8, 9.

Governance Constraint

Entity types must match canonical definitions. No entity type may be introduced without a corresponding Data Spec entry and canonical amendment. Each entity instance must have a unique ID for traceability. Entity removal must be logged

for the Operator Surface.

3.6 Data Spec Layer

Purpose: Canonical data definitions, configuration constants, and type schemas. This is the static truth layer — it defines what things *are* before they exist as runtime instances.

Responsibilities

- Petal type definitions (Guilt, Desire, Rage, Mercy) with associated effect values
- Thorn zone configuration (corruption-per-tick rate, sustained damage threshold and bonus)
- Altar configuration (redemption threshold for unlock)
- State variable schemas (types, ranges, defaults)
- Event definitions (loaded from or mirroring the Canonical Event Table)
- Ending type enum definitions
- Aesthetic configuration (color mappings, narrative text banks)

Interface

Direction	Description
Inputs	Canonical documents (spec, event table) — loaded at initialization
Outputs	Configuration data consumed by Entity Layer, Rules Engine, Event System, Render Layer
Dependencies	None (static data — leaf dependency)

Canonical References: All canonical documents.

Governance Constraint

The Data Spec Layer is the programmatic representation of canonical documents. Changes to this layer must follow the canonical amendment process. Hardcoded magic numbers in other modules are forbidden — all constants must be sourced from Data Spec. If a value appears in code that is not traceable to a Data Spec entry, it is a governance violation.

3.7 Render Layer

Purpose: Transforms game state into visual and textual output. This is the presentation surface the player sees.

Responsibilities

- Canvas or DOM-based rendering of game world (player, petals, thorns, altar, environment)
- HUD rendering (corruption meter, redemption meter, score readout)
- Narrative log display (rolling text log of event narrative outputs)
- Win/lose screen rendering
- Visual effects tied to state (e.g., darkening at high corruption, glow at high redemption)
- Input capture (keyboard events for player movement) forwarded to Entity Layer

Interface

Direction	Description
Inputs	Entity positions and states from Entity Layer; state values from State Manager; narrative text from Event System
Outputs	Visual display; input events forwarded to Entity Layer (player movement commands)

Direction	Description
Dependencies	State Manager (read-only for HUD); Entity Layer (read-only for positions); Event System (receives narrative dispatches)

Canonical References: Bloodrose Canonical Spec v0.1, Sections 12, 13.

Governance Constraint

The Render Layer must never modify game state. It is a pure consumer. Visual interpretation may be expressive (atmosphere, animation, color shifts) but must accurately represent the underlying state. The Render Layer must not suppress, delay, or fabricate narrative log entries — it displays exactly what the Event System dispatches, in the order dispatched.

3.8 Operator Surface Module

Purpose: The governed observation and control layer for system architects and operators. Sits alongside the Render Layer but serves a different audience — technical operators, not players.

Responsibilities

- Real-time state inspection (all canonical state variables, updated per tick)
- Event log with full metadata (EVT-ID, priority, source, timestamp, effects applied)
- Canonical compliance monitoring (unauthorized event detection, bounds violations, dependency violations)
- Entity inspection (instance IDs, positions, active/inactive status)
- Session summary generation
- Debug write controls (development builds only)

Interface

Direction	Description
Inputs	State from State Manager; events from Event System; entity data from Entity Layer
Outputs	Operator display (separate panel or overlay); session exports (JSON/CSV); compliance alerts
Dependencies	State Manager (read-only), Event System (read-only), Entity Layer (read-only)

Canonical References: Bloodrose Operator Surface Spec v0.1 (entire document).

Governance Constraint

Full specification lives in the Operator Surface Spec artifact. The Operator Surface consumes the same state as the Render Layer but displays it in raw, unglamoured form. Write controls are available in development builds only and permanently flag any affected session as **non-canonical**. In production builds, the Operator Surface is strictly read-only.

4. Module Interface Contracts

The following defines the formal APIs between modules. Each interface specifies the calling module, the providing module, the method signature, and the behavioral contract. These contracts are versioned and changes require architectural review.

4.1 State Manager API

```
getState(key: string): any
```

Returns the current value of a state variable. Pure read operation, no side effects. Returns `undefined` for unrecognized keys.

```
setState(key: string, value: any, source: string): void
```

Sets a state variable. The `source` parameter identifies the calling module for the audit trail. Validates bounds (clamps corruption and redemption to 0-100). Logs the mutation with timestamp, previous value, new value, and source. Emits change notification to all subscribers of the affected key.

```
getSnapshot(): StateSnapshot
```

Returns a serialized deep copy of all state variables at the current moment. The snapshot is immutable — modifications to the returned object do not affect internal state. Used for debugging, replay, and Operator Surface inspection.

```
subscribe(key: string, callback: function): unsubscribe
```

Registers a listener for changes to a specific state variable. Returns an unsubscribe function. Callbacks are invoked synchronously after the state change is applied and logged.

```
reset(): void
```

Resets all state to initial default values as defined in the Data Spec Layer. Only callable by the Game Shell. Logs the reset event.

4.2 Rules Engine API

```
evaluate(state: StateSnapshot, entities: EntityQuery): RuleResult[]
```

Evaluates all rules against the current state and entity positions. Returns an array of triggered rule results, each containing the rule ID, condition details, and associated event references. This method is a pure function — it produces no side effects and does not modify any external state.

4.3 Event System API

```
processEvents(ruleResults: RuleResult[]): void
```

Receives triggered rules from the Rules Engine. Resolves each to a canonical event via the Event Table registry. Sorts events by priority (Priority 0 first, ascending). Applies state effects through State Manager setters. Dispatches narrative text to the Render Layer. Logs all processed events.

```
registerEvent(eventDef: EventDefinition): void
```

Registers an event definition, typically called during initialization when loading from the Data Spec Layer. Each event must have a unique EVT-ID. Duplicate registrations are rejected and logged as warnings.

```
getEventLog(): EventLogEntry[]
```

Returns the full event log for the current session, ordered chronologically. Each entry contains: EVT-ID, tick number, priority, effects applied, narrative text dispatched, and source rule ID.

4.4 Entity Layer API

```
getEntities(type?: string): Entity[]
```

Returns all active entities, optionally filtered by entity type string. Returns an empty array if no entities match.

```
getEntity(id: string): Entity
```

Returns a specific entity by unique ID. Returns `null` if the entity does not exist or has been removed.

```
removeEntity(id: string): void
```

Removes an entity from the active set (e.g., petal consumed on collection). The entity is marked inactive, not deleted, to preserve audit trail. Logs the removal with entity ID, type, and tick number.

```
updatePosition(id: string, position: Vector2): void
```

Updates an entity's position. Primarily used for player movement in response to keyboard input. Validates that the entity exists and is active.

```
queryCollisions(entityId: string): Entity[]
```

Returns all active entities whose bounds overlap with the specified entity's bounds. Used by the Rules Engine for collision detection. Returns an empty array if no collisions are detected.

4.5 Data Spec API

```
getConfig(key: string): any
```

Returns a configuration value by dot-notation key path. Example:

`getConfig("petal.guilt.corruptionDelta")` returns `15`. Returns `undefined` for unrecognized keys.

```
getEntitySchema(type: string): EntitySchema
```

Returns the schema definition for an entity type, including required properties, default values, and bounds dimensions. Used by the Entity Layer during instantiation.

```
getEventDefinitions(): EventDefinition[]
```

Returns all canonical event definitions for registration with the Event System. Each definition includes EVT-ID, trigger conditions, priority, state effects, and narrative text.

4.6 Render Layer API

```
render(state: StateSnapshot, entities: Entity[]): void
```

Main render call, invoked once per tick. Draws the current frame based on entity positions and state values. Updates the HUD (corruption meter, redemption meter, score). This method must not modify any input data.

```
appendNarrative(text: string, eventId: string): void
```

Adds a narrative log entry to the rolling text display. The `eventId` associates the text with its source event for traceability. Entries are displayed in the order received and must not be reordered, suppressed, or modified.

```
showEndScreen(endingType: string, narrative: string): void
```

Displays the win/lose screen with the appropriate ending type and narrative text. Called once when `game_over` becomes true.

5. Data Flow Specification

The following defines the per-tick data flow through the module stack. Each step executes in strict sequence within a single tick. No parallel execution within a tick in v0.1.

Step	Phase	Module	Action
1	TICK	Game Shell	Signals a new tick to all modules. Increments the tick counter.
2	INPUT	Render Layer	Captures keyboard input from the player. Forwards movement commands to the Entity Layer. No state modification occurs.

Step	Phase	Module	Action
3	MOVE	Entity Layer	Updates the player entity's position based on the movement command received. All position changes are applied atomically.
4	EVALUATE	Rules Engine	Reads current state from the State Manager and entity positions from the Entity Layer. Evaluates all rules (collision detection, threshold checks, win/lose conditions). Returns an array of triggered conditions. Does not modify state.
5	PROCESS	Event System	Receives triggered conditions. Resolves to canonical events via the Event Table. Sorts by priority (Priority 0 first). Applies state effects through State Manager setter API. Dispatches narrative text to Render Layer.
6	RENDER	Render Layer	Reads updated state and entity positions. Draws the current frame. Updates HUD meters (corruption, redemption, score). Appends any new narrative

Step	Phase	Module	Action
			entries to the log.
7	OBSERVE	Operator Surface	Reads state, event log, and entity data. Updates operator display panels. Runs compliance checks (unauthorized events, bounds violations). Logs any alerts.
8	TERMINAL CHECK	Game Shell	Checks if <code>game_over</code> is true. If so: halts the tick loop, instructs Render Layer to display end screen, instructs Operator Surface to generate session summary.

Determinism Guarantee

Steps 2–7 execute in strict sequence within a single tick. No parallel execution within a tick in v0.1. This ensures deterministic behavior — the same input sequence will always produce the same state sequence, enabling replay debugging and regression testing against Phase 1.

6. Dependency Graph

The following table maps module dependencies. **R** indicates a read dependency (the row module reads from the column module). **W** indicates a write dependency (the row module writes to the column module). **I** indicates an instantiation-only dependency. — indicates no dependency. The dependency graph must remain a directed acyclic graph (DAG).

Module (row depends on column →)	Game Shell	State Mgr	Rules Eng	Event Sys	Entity Lyr	Data Spec	Render Lyr	Op Surface
Game Shell	—	I	I	I	I	I	I	I
State Manager	—	—	—	—	—	—	—	—
Rules Engine	—	R	—	—	R	—	—	—
Event System	—	W	R	—	—	R	W	—
Entity Layer	—	—	—	—	—	R	—	—
Data Spec	—	—	—	—	—	—	—	—
Render Layer	—	R	—	—	R	—	—	—
Operator Surface	—	R	—	R	R	—	—	—

Leaf Dependencies

The State Manager and Data Spec Layer are leaf dependencies — they depend on no other module. This makes them the foundation of the initialization sequence and the safest modules to extract first during Phase 1 → Phase 2 migration.

7. Module Lifecycle

Every module passes through four lifecycle stages: Initialization, Runtime, Session End, and Restart. The following defines the exact sequence for each stage.

7.1 Initialization Sequence

Modules must be initialized in strict order. Each step depends on the previous step completing successfully. If any module fails to initialize, the game must not start.

1. **Data Spec Layer** — Loads canonical configurations, entity schemas, event definitions, and all constants from canonical documents.
2. **State Manager** — Initializes with default state values sourced from Data Spec (corruption: 0, redemption: 0, score: 0, game_over: false, etc.).
3. **Entity Layer** — Instantiates all entities using schemas from Data Spec. Assigns unique IDs. Sets initial positions.
4. **Rules Engine** — Loads rule definitions. Binds to State Manager (read) and Entity Layer (read).
5. **Event System** — Registers all events from Data Spec / Canonical Event Table. Initializes one-shot tracking. Binds to State Manager (write) and Render Layer (narrative dispatch).
6. **Render Layer** — Initializes canvas/DOM rendering context. Binds input listeners for keyboard events. Prepares narrative log display.
7. **Operator Surface** — Initializes observation panels. Binds to State Manager, Event System, and Entity Layer for read-only observation. Starts session log.
8. **Game Shell** — All modules confirmed ready. Starts the game loop. First tick begins.

7.2 Runtime

All modules are active. The tick cycle executes per the Data Flow Specification (Section 5) on every frame. The Game Shell manages timing. Modules respond to

their designated tick-phase calls. This continues until `game_over` becomes true or the player initiates a pause/restart.

7.3 Session End

When `game_over` is set to true (by the Event System processing a terminal event):

9. **Game Shell** halts the tick loop. No further ticks are issued.
10. **Event System** processes the terminal event (EVT-ID with Priority 0), applies final state effects, dispatches terminal narrative text.
11. **Render Layer** displays the end screen with the appropriate ending type and narrative.
12. **Operator Surface** generates the session summary (total ticks, events fired, final state snapshot, compliance report).
13. All modules enter idle state, awaiting restart or session close.

7.4 Restart

When the player initiates a restart (or the Game Shell programmatically restarts):

14. **State Manager** resets all state to default values via `reset()`.
 15. **Entity Layer** re-instantiates all entities to initial positions and active status.
 16. **Event System** clears the event log and resets all one-shot event trackers.
 17. **Render Layer** clears the display and narrative log. Re-renders the initial frame.
 18. **Operator Surface** archives the previous session log and starts a new session log.
 19. **Game Shell** resumes the tick loop. A fresh session begins.
-

8. Migration Path: Phase 1 → Phase 2

The following defines the practical step-by-step process for decomposing the single-file Phase 1 prototype into the modular architecture defined in this document. Each step must be validated before proceeding to the next. The goal is behavioral parity — Phase 2 must produce identical deterministic output to Phase 1 for any given input sequence.

Step 1: Extract Data Spec

Move all hardcoded constants out of the monolithic file: corruption deltas, threshold values, petal type definitions, thorn zone rates, altar configuration, ending conditions. Place them in a dedicated data module or JSON configuration file. Replace all inline references to magic numbers with calls to the Data Spec API. Validate: the game still runs identically with all values sourced from Data Spec.

Step 2: Extract State Manager

Isolate all state variables into a dedicated state object with getter/setter methods. Replace every direct state mutation (`corruption += 15`) with setter calls (`setState("corruption", getState("corruption") + 15, "eventSystem")`). Add bounds clamping. Add mutation logging. Validate: state bounds are enforced, all mutations are logged.

Step 3: Extract Entity Layer

Create entity classes or factory functions with unique IDs, type identifiers, position vectors, and bounding dimensions. Replace all inline entity handling (array iteration, position checks) with Entity Layer queries. Validate: all entities are addressable by ID, collision queries return correct results.

Step 4: Extract Rules Engine

Move all collision detection and threshold checking into a dedicated evaluation module. The Rules Engine must return results as data — it must not call any state setters. Replace inline `if (corruption >= 50) { ... }` blocks with rule evaluation

results. Validate: the Rules Engine produces correct condition flags for known state inputs without modifying state.

Step 5: Extract Event System

Move event definitions into a registry keyed by EVT-ID. Move all state-effect application code into the Event System's `processEvents` method. Wire priority ordering. Wire one-shot tracking. Validate: every event maps to a canonical EVT-ID, priority ordering is deterministic, one-shot events fire exactly once.

Step 6: Extract Render Layer

Isolate all drawing code, HUD rendering, and narrative log display into a render module that consumes state and entity data as read-only inputs. The Render Layer must not contain any game logic, threshold checks, or state mutations. Validate: the render module draws correctly from external data, game logic functions without rendering.

Step 7: Wire Operator Surface

Build the Operator Surface module as a separate observation layer. Bind it to the State Manager (subscribe to state changes), Event System (receive event log), and Entity Layer (query entity states). Validate: the Operator Surface accurately reflects all game state and events in real-time without interfering with gameplay.

Step 8: Validate

Run the game with the full modular architecture. Record a complete playthrough. Compare behavior tick-for-tick against a Phase 1 prototype recording to confirm identical deterministic output. Any divergence is a regression and must be resolved before Phase 2 is declared complete.

Migration Acceptance Criteria

Phase 2 is not complete until a recorded Phase 1 session can be replayed against Phase 2 modules with zero state divergence. Behavioral parity is the acceptance gate. Visual differences (rendering improvements) are permitted, but state history must be identical.

9. Testing Strategy

9.1 Unit Testing

Each module is tested in isolation with controlled inputs and verified outputs.

Module	Test Focus	Key Test Cases
State Manager	Bounds clamping, mutation logging, snapshot accuracy	Set corruption to 150 → verify clamped to 100. Set redemption to -5 → verify clamped to 0. Verify snapshot matches internal state. Verify mutation log records source module.
Rules Engine	Condition evaluation with known state inputs	Inject state with corruption=50 → verify threshold flag triggers. Place player on petal position → verify collision detected. Place player off all entities → verify no triggers.
Event System	Priority ordering, one-shot tracking, effect application	Submit Priority 0 and Priority 3 events → verify Priority 0 processes first. Fire a one-shot event twice → verify second fire is suppressed. Verify state effects applied through State Manager API.
Entity Layer	Instantiation, removal, collision queries	Instantiate entity → verify unique ID assigned. Remove entity → verify marked inactive. Query collisions with overlapping bounds → verify correct results.

Module	Test Focus	Key Test Cases
Data Spec	Configuration loading, schema validation	Load config → verify all required keys present. Request entity schema → verify required properties defined. Verify no undefined values for canonical keys.

9.2 Integration Testing

- **Full tick cycle:** Inject known input, advance one tick through the entire module stack, verify state output matches expected values.
- **Event chain testing:** Trigger a sequence of petal collections and thorn exposures, verify cumulative state (corruption, redemption, score) matches canonical rules.
- **Terminal condition testing:** Drive state to each terminal condition (corruption $\geq$ 100, redemption endings) and verify the correct ending type triggers with correct narrative.
- **Compliance testing:** Verify no unauthorized events fire, no state bounds are violated, no unregistered EVT-IDs appear in the event log.

9.3 Regression Testing

- **Phase 1 parity:** Replay a recorded Phase 1 session input sequence against Phase 2 modules and verify identical state history at every tick.
 - **Canonical compliance:** Automated scan that every event registered in the Event System maps to a valid EVT-ID in the Canonical Event Table. Any orphaned or unregistered event is a test failure.
-

10. Error Handling and Fault Isolation

Errors must be contained, logged, and surfaced without crashing the game loop. The architecture treats errors as data to be observed and acted upon — never as silent failures.

Error Source	Behavior	Severity
Module-level exception	Game Shell catches the exception. Logs it with module identity and tick number. Game loop continues. Operator Surface displays the error.	ERROR
Invalid state mutation (out of bounds, wrong type)	State Manager rejects the mutation. Logs a warning with the attempted value, the calling module, and the valid range. State is unchanged.	WARNING
Unregistered event attempts to fire	Event System refuses to process the event. Logs a CRITICAL alert with the unrecognized EVT-ID and the triggering rule. No state effects applied.	CRITICAL
Missing entity data in Render Layer	Render Layer gracefully degrades — renders a placeholder or skips the missing entity. Logs a warning. Does not crash the render pass.	WARNING
Operator Surface observation failure	Operator Surface logs the failure internally. Does not affect game execution. The Operator Surface is non-critical to gameplay.	WARNING

Build Mode Behavior

Production builds: Errors are silent to the player but visible on the Operator Surface. The game loop never halts due to non-terminal errors.

Development builds: Errors may optionally pause the game loop for investigation. CRITICAL alerts trigger an immediate pause. All errors are displayed in a development console overlay in addition to the Operator Surface.

11. Integration Points with Other Artifacts

This architecture document does not exist in isolation. It connects to the broader Bloodrose production suite through explicit integration points. Each connection is defined by which module provides or consumes data and which artifact governs the contract.

Artifact	Integration Module(s)	Relationship
Canonical Event Table (Artifact 1)	Event System, Data Spec Layer	The Event System module loads its event registry from the Event Table. Every event must have a matching EVT-ID. The Data Spec Layer stores event definitions as the programmatic representation of the Event Table.
Contractor Contract (Artifact 2)	All modules	Contractors implementing modules must follow the interface contracts defined in this document. Module boundaries define deliverable scope. The Contractor Contract governs contributor

Artifact	Integration Module(s)	Relationship
		behavior and acceptance criteria.
Operator Surface Spec (Artifact 3)	Operator Surface Module	The Operator Surface Module implements the specification from Artifact 3. It consumes data from State Manager, Event System, and Entity Layer as defined in this architecture.
Drift Detection Harness (Artifact 5)	Operator Surface, State Manager	The harness tests this architecture's modules against canonical constraints. Compliance data flows from the Operator Surface Module to the harness. State snapshots from the State Manager provide the data substrate for drift analysis.
Story Forge Integration Layer (Artifact 6)	Data Spec Layer, Event System	Future integration point. The Data Spec Layer and Event System provide the hooks for Story Forge's governed narrative pipeline. Narrative text banks in Data Spec and event dispatch in Event System are the connection surfaces.
Render Pipeline Spec (Artifact 7)	Render Layer	The Render Layer module implements the rendering specification from Artifact 7. Visual effects, HUD layout, narrative log presentation, and end screen composition are governed by the Render Pipeline Spec.

12. Governance Constraints

The following architectural rules are non-negotiable. They are structural invariants that must hold true at all times, in all builds, across all contributors. Violations constitute architectural regressions and must be resolved before any affected code is merged.

Non-Negotiable Architectural Rules

1. **No direct internal state access.** No module may access another module's internal state directly. All communication goes through the defined APIs specified in Section 4.
2. **Single source of truth.** The State Manager is the single source of truth for all mutable game state. No shadow state, no local caches of mutable values, no module-local copies of state variables.
3. **Evaluate/Execute separation.** The Rules Engine evaluates; the Event System executes. This separation is mandatory for auditability. No module may combine rule evaluation and state mutation in a single operation.
4. **Render Layer purity.** The Render Layer is a pure consumer. It never modifies game state. It never suppresses, delays, or fabricates narrative log entries.
5. **Interface versioning.** All module interfaces are versioned. Changes to interfaces require architectural review and a formal amendment to this document.
6. **Directed acyclic dependencies.** No circular dependencies. The dependency graph (Section 6) must remain a directed acyclic graph (DAG) at all times.
7. **Canonical traceability.** Every event, entity type, threshold value, and configuration constant must be traceable to a canonical document. Hardcoded

magic numbers are forbidden.

8. **Suggestion is not truth.** Architectural proposals, contributor suggestions, and LLM-generated modifications must be validated against this document before implementation. This document is the authority.

13. Future Considerations (Phase 3+)

The following are directional notes on where the architecture may evolve beyond Phase 2. These are not commitments — they are architectural affordances that the current design enables without requiring.

- **Plugin Interface:** The Entity Layer and Event System may expose governed plugin hooks for extensions. New entity types and event definitions could be loaded at runtime through a validated plugin manifest, subject to canonical compliance checks before activation.
- **Networked State:** The State Manager's snapshot/restore capability enables future networked or multiplayer state synchronization. The deterministic tick model means state can be reconstructed from an input log, supporting lockstep or rollback networking models.
- **Cross-Media Render Targets:** The Render Layer's complete separation from game logic means alternative render targets — visual novel renderer, cinematic renderer, accessibility-focused text renderer — can be swapped in without touching game logic, rules, or event processing.
- **Automated Governance:** The Operator Surface and Drift Detection Harness may be automated to run as CI/CD pipeline gates. Every pull request could be validated against canonical compliance constraints before merge, with automated session replay regression checks.

- **Save/Load System:** The State Manager's serialization support and the Entity Layer's ID-based tracking provide the foundation for a save/load system. A full session state can be captured as a snapshot and restored to resume play.

14. Amendment Log

All changes to this document must be recorded in the amendment log with a unique amendment ID, date, description, author, and approval status. This log is the version history of the architecture itself.

Amendment ID	Date	Description	Author	Approved By
AMD-000	April 21, 2026	Initial Modular Architecture Document created. Defines all eight modules, interface contracts, data flow specification, dependency graph, lifecycle stages, migration path, testing strategy, error handling, artifact integration points, and governance constraints.	System Architect	Pending Review

